

Race Conditions — Study Guide

Race conditions are the most "systems-thinking" vulnerability class in this entire series — closely related to business logic flaws, they don't involve malformed input, injection payloads, or broken crypto at all. Instead, they exploit **timing**: what happens when an application processes two requests touching the same data at almost exactly the same moment. This guide also directly revisits your Authentication guide's 2FA bypass technique and your Sessions guide's cookie-swap attack — race conditions give you a *third*, purely timing-based way to defeat the same 2FA logic.

1. What a Race Condition Actually Is

A race condition occurs when a website processes requests **concurrently** without adequate safeguards, letting multiple threads interact with the same data at the same time — causing a **collision** that produces unintended behavior. An attacker deliberately times requests to trigger that collision on purpose.

The core concept: the "race window" — the brief period during which a collision is possible. This might be the fraction of a second between an application checking a condition and then acting on it — sometimes just milliseconds, occasionally even shorter.

2. Limit Overrun Race Conditions (The Classic Pattern)

The most well-known race condition type lets you **exceed a limit** the application's business logic tries to enforce.

Example — a one-time discount code:

Normal (sequential) flow:

1. Check the code hasn't already been used
2. Apply the discount to the order total
3. Update the database to mark the code as used

If you try to reuse it afterward, step 1 correctly blocks you.

The race: if you submit **two requests using the code at almost exactly the same time**, both requests can pass step 1's check **before either one has completed step 3** — the application briefly transitions through a **sub-state**: a state it enters and exits again before request processing finishes, during which the "already used" flag hasn't been written yet. Both requests apply the discount, and you've used a one-time code twice.

This is a specific case of a broader class called TOCTOU — Time-Of-Check to Time-Of-Use — the general pattern where a condition is checked, then acted upon, with a gap between the two where reality can change out from under the check.

Real-world variations of this exact pattern:

- Redeeming a gift card multiple times
- Rating a product multiple times
- Withdrawing/transferring more cash than your actual account balance
- Reusing a single CAPTCHA solution
- Bypassing an anti-brute-force rate limit itself (using the rate limiter's own check-then-block gap against it)

3. Detecting and Exploiting Limit Overruns — The Basic Method

1. Identify a single-use or rate-limited endpoint with genuine security/business impact
2. Fire multiple requests at it in rapid succession, attempting to overrun the limit

The core challenge: getting two requests' race windows to actually overlap. Even sending requests "at the same time" from your side doesn't guarantee they arrive and get processed simultaneously — unpredictable network jitter routinely throws off naive attempts.

a) Burp Repeater's Parallel Request Techniques

Modern Burp Suite versions solve this directly, automatically choosing the right technique for the target's HTTP version:

HTTP/1 → "last-byte synchronization" – send all requests almost complete, withholding the final byte of each, then

```
release all final bytes simultaneously
HTTP/2 → "single-packet attack" – bundle 20-30 requests into a
        SINGLE TCP packet, completely eliminating network jitter
        as a variable, since the server receives them as one unit
```

The single-packet attack (first presented by PortSwigger Research at Black Hat USA 2023) is genuinely the state of the art here — worth knowing it exists as a named, specific technique when this topic comes up in an interview.

b) Turbo Intruder — For More Complex Attacks

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=1,
                           engine=Engine.BURP2)

    # queue 20 requests into a named "gate"
    for i in range(20):
        engine.queue(target.req, gate='1')

    # release all requests in that gate simultaneously
    engine.openGate('1')
```

Turbo Intruder (a Burp extension, requires some Python) is suited to attacks needing retries, staggered timing, or very large request volumes — beyond what Repeater's built-in grouping conveniently handles. This is directly relevant to your Authentication guide's mention of Turbo Intruder for automating multi-step 2FA brute-force flows — same tool, same underlying reason (precise, scriptable request timing/sequencing).

4. Hidden Multi-Step Sequences — Beyond Simple Limit Overruns

This is where race conditions get genuinely interesting, and where they connect directly to earlier guides. A **single HTTP request** on the backend can silently trigger an entire multi-step sequence — transitioning the application through hidden **sub-states** before the response is returned.

The headline example — a race variation of the 2FA bypass from your Authentication guide:

```
session['userid'] = user.userid
if user.mfa_enabled:
    session['enforce_mfa'] = True
    # generate and send MFA code
    # redirect to MFA code entry form
```

Notice this is a **multi-step sequence within a single request** — and critically, there's a sub-state where the user has a **valid logged-in session**, but MFA enforcement hasn't been set yet. If an attacker fires the login request **and** a request to a sensitive authenticated endpoint **at almost exactly the same time**, the sensitive request might land in that narrow gap — before `enforce_mfa` gets set — bypassing 2FA entirely through pure timing, without needing your Sessions guide's cookie-manipulation trick or your Authentication guide's "navigate directly to a logged-in page" trick at all. **Three completely different techniques, same underlying flaw category (broken 2FA enforcement), from three different guides in this series.**

5. The Predict → Probe → Prove Methodology

PortSwigger Research's structured approach for finding hidden race conditions systematically, rather than randomly firing requests everywhere:

Step 1 — Predict Potential Collisions

Testing every endpoint isn't practical — narrow down with two questions:

- Is this endpoint security-critical? (skip anything without real impact)
- Is there genuine collision potential? – you need two+ requests that operate on the SAME underlying record

Example distinguishing collision potential — password reset implementations:

Implementation A: parallel resets for TWO DIFFERENT users
→ touches two SEPARATE records → unlikely to collide

Implementation B: both requests write to the SAME session/record
(e.g., a shared "pending reset" state keyed by session, not by user)
→ genuine collision potential exists

Step 2 — Probe for Clues

1. Benchmark NORMAL behavior first – send your request group SEQUENTIALLY (separate connections) and record what happens
2. Send the SAME group again, this time in PARALLEL (single-packet attack / Turbo Intruder)
3. Compare – ANYTHING that deviates from the benchmark is a clue: a changed response, a second-order effect (different email content sent afterward, a visible behavior change on next login)

This benchmark-then-compare discipline is exactly the same "establish a baseline, then look for deviation" methodology from your tcpdump guide's traffic-baselining discussion and your SQLi guide's boolean-blind detection technique — a recurring pattern across this whole series: you can't recognize an anomaly without first knowing what normal looks like.

Step 3 — Prove the Concept

- Strip down to the minimal set of requests that still reproduces the effect
- Confirm you can replicate it reliably, not just once by luck
- Think of the race condition as a STRUCTURAL WEAKNESS in the application's state machine, not an isolated one-off bug – the same underlying flaw may be exploitable via several different request combinations, not just the first one you found

6. Multi-Endpoint Race Conditions

Some race conditions require hitting **multiple different endpoints simultaneously**, rather than the same endpoint repeatedly.

Example — a variation of the classic "add items to cart after paying" logic

flaw: if payment validation and order confirmation happen within the processing of a *single* request, there's a race window between "payment validated" and "order finally confirmed" — sending a request to **add more items to the basket** timed into that exact window can let those extra items ride along on an order that's already been marked as paid.

Aligning Multi-Endpoint Race Windows — Two Common Obstacles

a) Network architecture delays — a front-end server establishing a new backend connection introduces latency that, importantly, usually delays *all* parallel requests roughly equally, keeping them in sync with each other even though everything is slower overall.

b) Endpoint-specific processing delays — different endpoints inherently take different amounts of time to process, which *does* throw off alignment, since it's not a uniform delay across your request group.

Connection warming — distinguishing the two:

Add an inconsequential GET request (e.g., the homepage) to the START of your request group, sent via a single connection.

If the FIRST real request in the group still takes longer, but everything AFTER it lands within a tight window → the delay was just connection-establishment overhead, safe to ignore and proceed

If timing STAYS inconsistent even with warming → endpoint-specific processing delay is genuinely interfering with your attack

Abusing rate/resource limits as a workaround: if a genuine processing delay can't be smoothed out, one clever technique is deliberately triggering the target's **own rate-limiting feature** with a burst of throwaway requests first — the resulting server-side slowdown can be exploited to create the precise timing gap your actual attack requests need, all while still fitting within a single-packet attack. A nice example of turning a security control into the very thing that enables the attack.

7. Single-Endpoint Race Conditions — Different Values, Same Endpoint

Sending parallel requests to a **single** endpoint but with **different parameter values** can trigger powerful collisions when the endpoint's logic touches shared/session state.

Example — a password reset mechanism storing state in the session:

Two parallel requests, same session, different usernames:
Request A: reset for "attacker"
Request B: reset for "victim"

Possible resulting collision (order-dependent):
session['reset-user'] = victim (from Request B)

```
session['reset-token'] = 1234      (the token generated for  
                                   Request A's attacker-initiated  
                                   reset, if it completes AFTER  
                                   B sets the username but BEFORE  
                                   A's own token write happens)
```

Net result: the session now believes the "victim" is the user being reset, but the VALID reset token got emailed to the ATTACKER

This requires the operations to land in a very specific order — often needing multiple attempts or genuine luck to hit the exact right interleaving, but it's a devastating account-takeover primitive when it lands. **Email-based operations are a particularly good target for this class**, since emails are frequently dispatched in a background thread *after* the HTTP response is already returned — widening the effective race window well beyond the request/response cycle itself.

8. Session-Based Locking — A Mechanism That Can Mask Real Bugs

Some frameworks (PHP's native session handler is the classic example) only process **one request per session at a time** by default, as a built-in safeguard against accidental data corruption.

Why this matters for testing, not just for defense: if every request in your test group appears to be processed strictly sequentially no matter what timing technique you throw at it, this is a strong signal that session-level locking is silently serializing your requests — **masking** a genuinely exploitable race condition underneath. The fix for your *testing* methodology: **send each request using a different session token**, sidestepping the lock and revealing the underlying (potentially still vulnerable) behavior.

9. Partial Construction Race Conditions

Many applications build objects across **multiple separate steps** — creating a database row in one statement, then setting a field like an API key in a second, separate statement. This leaves a genuine window where the object **exists but is incompletely initialized**.

The exploitation angle: if you can supply an input that matches whatever the **uninitialized** value looks like (often an empty string, or `null`), and that

comparison is used as a security check, you can potentially authenticate during that exact window using essentially "nothing" as your credential.

Framework-specific syntax tricks for supplying exactly this kind of value (PHP):

```
param[]=foo           → param = ['foo']  
param[]=foo&param[]=bar → param = ['foo', 'bar']  
param[]               → param = []      (an empty array, not a string –  
                                         useful for defeating a naive  
                                         "is this string empty" check  
                                         via a type mismatch instead)
```

Ruby on Rails equivalent — a parameter with a key but no value:

```
param[key] → server-side: {"param"=>{"key"=>nil}}
```

Practical attack shape:

```
GET /api/user/info?user=victim&api-key[]=
```

Sent during the exact window where the victim's own API key hasn't been written to the database yet, this can potentially match against the still-uninitialized (empty/null) value, authenticating as the victim with essentially zero real credential. The password-equivalent version of this is harder, since passwords are hashed — you'd need to find an input that happens to *hash* to whatever the uninitialized-value's stored form actually is, rather than just matching a raw empty string.

10. Time-Sensitive Attacks — When It's Not Really a Race Condition At All

Sometimes the precise-timing delivery techniques from this guide reveal a **different** vulnerability entirely — not a true race condition, but a token-generation weakness that timing just happens to expose.

Example: if a password reset token is generated using a **high-resolution timestamp** instead of a genuine cryptographically secure random string (directly violating your Sessions guide's Section 2 principle on session/token quality), you can time two separate password-reset requests — for two

different users — to land within the same timestamp tick, causing both to receive the **identical** reset token. This isn't really exploiting a race in application logic — it's exploiting predictable token generation, with precise request timing as the delivery mechanism that makes the flaw observable at all.

11. Prevention — Eliminating Sub-States, Not Trying to Detect Them

The core defensive principle: once a single request can transition an application through invisible sub-states, reliably predicting and defending against every possible interleaving becomes impractical. The real fix is **removing the sub-states from sensitive endpoints entirely**, not trying to detect collisions after the fact:

- Avoid mixing data across different storage locations for a single logical operation
- Make sensitive state changes ATOMIC using the datastore's own concurrency features — e.g., a single database TRANSACTION that checks payment validity AND confirms the order together, rather than as two separate, individually-interruptible steps
- Use datastore integrity constraints (column uniqueness constraints) as defense-in-depth on top of atomicity, not as a substitute for it
- NEVER use one storage layer to try to secure another — e.g., sessions are NOT a suitable mechanism for preventing a limit-overflow attack against the underlying database
- Ensure session-handling frameworks keep session state internally consistent — updating session variables ONE AT A TIME instead of as a single atomic batch is a tempting "optimization" that's genuinely dangerous; this applies to ORMs too, since they often hide transaction handling from the developer entirely, silently taking on (or failing to take on) that responsibility
- In some architectures, avoiding server-side state entirely and pushing state to the CLIENT (e.g., via JWTs, per your JWT guide) is a legitimate architectural option — worth explicitly noting this trades the race-condition risk for the JWT-specific risks covered in that earlier guide instead, not eliminating risk outright

12. Practical Lab Example

PortSwigger's Web Security Academy has a dedicated race condition lab set, directly built around the exact techniques in this guide — including labs specifically reproducing the 2FA-bypass-via-race scenario from Section 4, which is worth doing immediately after revisiting your Authentication guide's 2FA bypass lab, since seeing

the SAME underlying vulnerable pattern defeated via THREE different techniques (direct navigation, cookie-swap, and now pure timing) across your last two guides really cements how these logic-flaw categories relate to each other.

Practice flow:

1. Start with a limit-overflow lab - use Burp Repeater's "Send group in parallel" option to practice the single-packet attack directly
2. Move to a hidden multi-step sequence lab (2FA race) - apply the Predict → Probe → Prove methodology explicitly
3. Try a multi-endpoint race lab, practicing connection warming to distinguish network delay from processing delay

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-362 (Concurrent Execution using Shared Resource with Improper Synchronization, "Race Condition"), CWE-367 (Time-of-check Time-of-use, TOCTOU)
OWASP Top 10:2025	A06:2025 Insecure Design (race conditions are explicitly a design-level state-machine flaw, not an implementation bug you can patch with input filtering) — some instances also touch A01:2025 Broken Access Control when the collision grants unauthorized access (Section 4/7's examples)
CVSS	Highly context-dependent — a discount-code limit overrun might be Low/Medium, while a 2FA-bypass or account-takeover race (Sections 4/7) can be Critical
Cyber Kill Chain	Exploitation — a race-condition-based auth bypass is functionally identical in Kill Chain terms to the JWT/session/2FA bypasses from earlier guides, just reached through timing instead of forgery/tampering
MITRE ATT&CK	No dedicated technique ID — race conditions are a root-cause vulnerability class that <i>enables</i> whichever specific ATT&CK technique the resulting access maps to (T1078 Valid Accounts, if 2FA/auth is bypassed)

Quick Reference Cheat Sheet

Race window = the brief period a collision is possible

Limit overrun (TOCTOU): check → act → update, race the gap between them

Hidden multi-step: ONE request internally transitions through sub-states

Multi-endpoint: race requests across DIFFERENT endpoints touching the same record
Single-endpoint: race the SAME endpoint with DIFFERENT parameter values
Partial construction: exploit an object's temporarily-uninitialized field
Time-sensitive: exploit predictable (non-random) token generation via precise timing

Tools:

Burp Repeater → "Send group in parallel" (single-packet attack / last-byte sync)

Turbo Intruder → scriptable, complex/staggered timing, gate-based grouping

Methodology: Predict (is it critical + collision-possible?) →

Probe (benchmark sequential, then compare to parallel) →

Prove (minimize, confirm reliable reproduction)

Masking signal: all requests processing strictly sequentially despite
parallel delivery → try a DIFFERENT session token per request